

CSE-4801 Compiler Design

Grammars & expressions

Israt Binte Habib

Lecturer

Department of CSE, IIUC

Language

- An alphabet is any finite set of symbols. $\Sigma = \{0,1\}$
- A string over an alphabet is a finite sequence of symbols drawn from that alphabet. $W = \{001100, 010\}$
- A language is any countable set of strings over some fixed alphabet. $L = \{ \text{Set of all strings over } (a,b) \}$

Context Free Grammar

- A grammar naturally describes the hierarchical structure of most programming language constructs
- A context-free grammar(CFG) is a grammar where each production has the form $A \rightarrow w$, where A is a nonterminal and w is a string of terminals and nonterminals.
- A context-free grammar has four components:
 1. A set of terminal symbols T , sometimes referred to as tokens.
 2. A set of nonterminals N , sometimes called “syntactic variables.”
 3. A set of productions P , where each production consists of a nonterminal, called the head or left side of the production, an arrow, and a sequence of terminals and/or nonterminals, called the body or right side of the production.
 4. A designation of one of the nonterminals as the start symbol S .

CFG

- Identify the terminals, non-terminals, start variable from the following grammar:
- $E \Rightarrow E + T \mid T$
 $T \Rightarrow T * F \mid F$
 $F \Rightarrow (E) \mid \text{id}$

Ans:

$N = \{E, T, F\}$

$T = \{+, *, (,), \text{id}\}$

$S = \{E\}$

CFG

- CFG for $L = \{0^n 1^n \mid n \geq 1\}$

$S \longrightarrow 01 \mid 0S1$

$N=2, S=0S1=0 \ 01 \ 1$

$N=3 \ S=0S1=0 \ 0S1 \ 1 = 00 \ 01 \ 11$

- $L = \{a^n \mid n \geq 0\}$ $s = \{\epsilon, a, aa, aaa, \dots\}$

$S \Rightarrow \epsilon \mid aS$

$n=4, S = aS = a \ aS = aa \ aS = aaa \ aS = aaaaS = aaaa. \ \epsilon = aaaa$

Assignment: CFG Construction

- $L = \{a^n \mid n \geq 1\}$
- $L = \{\text{set of all strings over } (a,b)\}$
- $L = \{\text{set of all strings of } (a,b) \text{ which length at least } 2\}$
- $L = \{\text{set of all strings start with } a\}$
- $L = \{\text{set of all strings ends with } b\}$
- $L = \{\text{set of all strings start with } a \text{ and ends with } b\}$
- $L = \{\text{set of all strings starts and ends with same symbol}\}$
- $L = \{\text{set of all strings of } (0,1) \text{ of even length}\}$
- $L = \{\text{set of all strings of } (0,1) \text{ of odd length}\}$

Terminals and Non-terminals

- A terminal is a symbol which does not appear on the left-hand side of any production. A grammar contains a set of terminal symbols (tokens) such as the plus sign, +, the times sign, *, and other tokens defined by the lexical analyzer such as *Identifiers*
- Nonterminals are the non-leaf nodes in a parse tree.
- $\langle \text{digit} \rangle ::= '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9'$
 $\langle \text{integer} \rangle ::= ['-'] \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \}$

In this example, the symbols (-,0,1,2,3,4,5,6,7,8,9) are terminal symbols and $\langle \text{digit} \rangle$ and $\langle \text{integer} \rangle$ are nonterminal symbols.

Useless Non-terminals

- Non-terminals that are unreachable (from the start symbol) or that derive no terminal string are considered useless.
- Consider $S \rightarrow AB \mid x$, $B \rightarrow b$, $A \rightarrow aA$, $C \rightarrow d$
Which non-terminals are unreachable? Which derive no terminal string?

Context-Sensitive Grammars

- A context-*sensitive* grammar (CSG) is a grammar where each production has the form

$$wAx \rightarrow wyx,$$

where w and x are strings of terminals and nonterminals and y is also a string of terminals.

- In other words, the productions give rules saying "if you see A *in a given context*, you may replace A by the string y ."

Linear grammars

The Linear Grammars are either left or right:

Right Linear Grammars:

Rules of the forms

$$A \rightarrow \varepsilon$$

$$A \rightarrow a$$

$$A \rightarrow aB$$

Example:

$$S \rightarrow aS \mid bA$$

$$A \rightarrow cA \mid \varepsilon$$

A,B: variables and
a: terminal

Left Linear
Grammars:

Rules of the forms

$$A \rightarrow \varepsilon$$

$$A \rightarrow a$$

$$A \rightarrow Ba$$

Example:

$$C \rightarrow Bc$$

$$B \rightarrow Ab$$

$$A \rightarrow a$$

A,B: variables and
A: terminal

Example

- Grammar for expressions consisting of lists of digits separated by plus or minus signs can be written as:

$\text{list} \longrightarrow \text{list} + \text{digit} \mid \text{list} - \text{digit} \mid \text{digit}$

$\text{digit} \longrightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

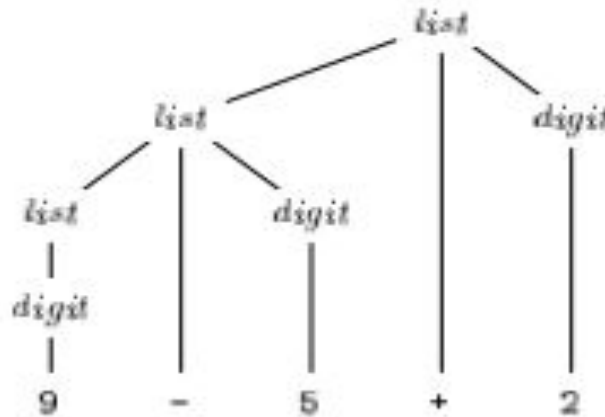
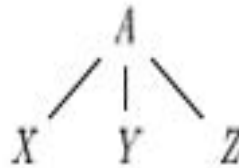
Derivations

- A grammar derives strings by beginning with the start symbol and repeatedly replacing a nonterminal by the body of a production for that nonterminal.
- For example, we can deduce that 9-5+2 is a list as follows

list $\longrightarrow$ list + digit
 $\longrightarrow$ list - digit + digit
 $\longrightarrow$ digit - digit + digit
 $\longrightarrow$ 9 - digit + digit
 $\longrightarrow$ 9 - 5 + digit
 $\longrightarrow$ 9 - 5 + 2

Parse Trees

- A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. If nonterminal A has a production $A \rightarrow XYZ$, then a parse tree



Parse tree for $9-5+2$

Ambiguity

- A grammar can have more than one parse tree generating a given string of terminals. Such a grammar is said to be ambiguous.
- $\text{string} \rightarrow \text{string} + \text{string} \mid \text{string} - \text{string} \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

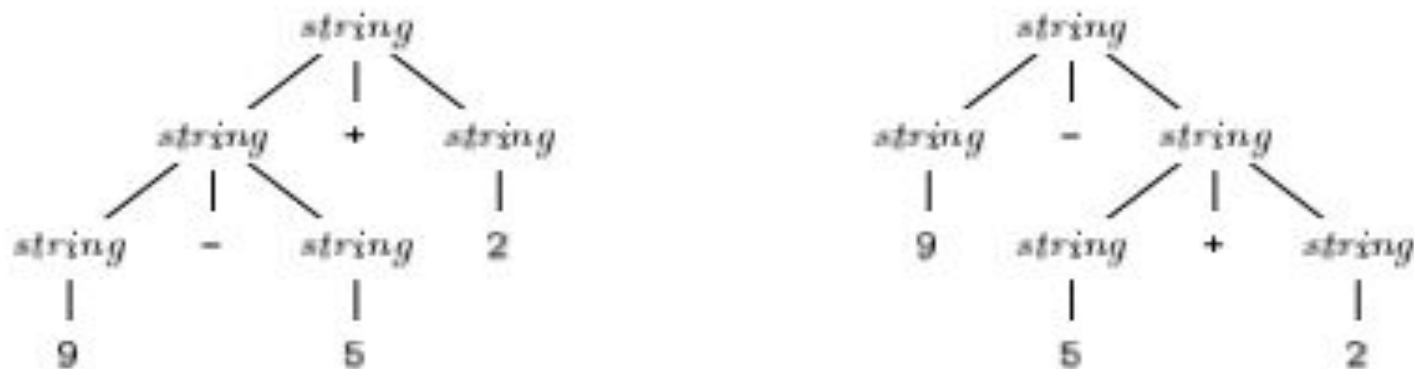


Figure 2.6: Two parse trees for 9-5+2

Associativity of Operators

- If an operand has operators on both sides, the side on which the operator takes this operand is decided by the associativity of those operators.
- $9+5+2$ is equivalent to $(9+5)+2$ and $9-5-2$ is equivalent to $(9-5)-2$.
- When an operand like 5 has operators to its left and right, conventions are needed for deciding which operator applies to that operand.
- In most programming languages the four arithmetic operators, addition, subtraction, multiplication, and division are left-associative. Some common operators such as exponentiation are right-associative.
- Strings like $a=b=c$ with a right-associative operator are generated by the following grammar:

right $\Rightarrow$ letter = right | letter

letter $\Rightarrow$ a | b | z

Associativity of Operators

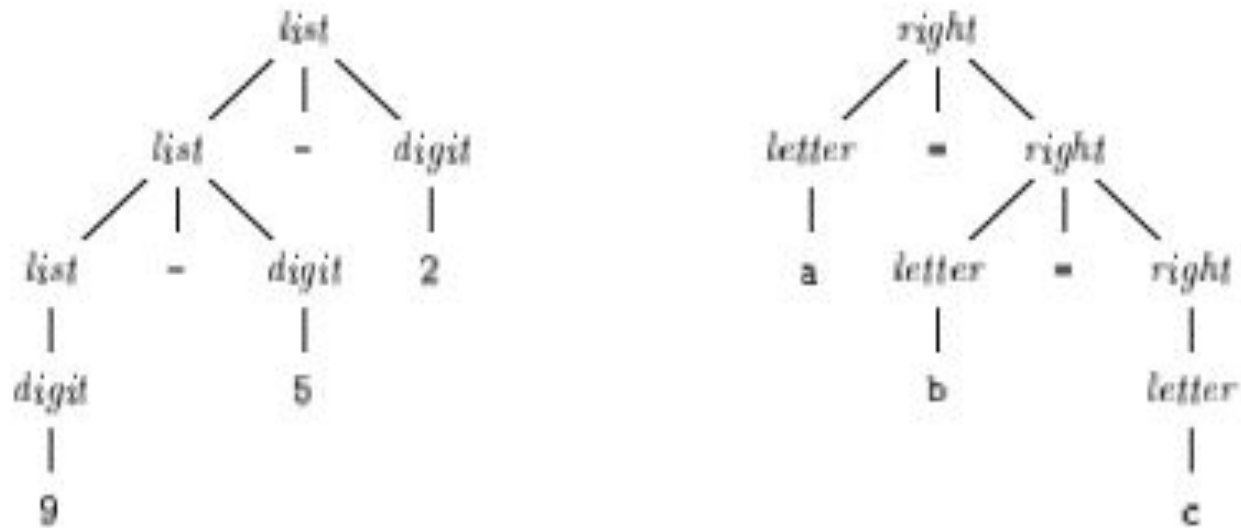


Figure 2.7: Parse trees for left- and right-associative grammars

Precedence of Operator

- If two different operators share a common operand, the precedence of operators decides which will take the operand.
- We say that $*$ has higher precedence than $+$ if $*$ takes its operands before $+$ does.
- In ordinary arithmetic, multiplication and division have higher precedence than addition and subtraction.
- Therefore, 5 is taken by $*$ in both $9+5*2$ and $9*5+2$; i.e., the expressions are equivalent to $9+(5*2)$ and $(9*5)+2$, respectively.